

Signed approvals: making Reth refuse what OPS did not approve

The proxy decides before execution. Execution can diverge. This PoC moves the final check inside the block producer.

Worktree	ops-reth-signed-approvals	Branch	poc/ops-reth-signed-approvals
Base	OPS 7172795 · Reth v2.5.2 5a6940e	Measured on	Ivan's MacBook — Apple M2 Max, 32 GB

The whole report in one table

Problem	OPS approves a transaction from a preflight trace. Another transaction — even one earlier in the <i>same block</i> — can change the state in between, so the approved call lands somewhere it was never approved to go.
Why not fix it in the proxy	The proxy is outside the thing that commits state. Re-tracing, killing the cache and bumping fees shrink the window; they never make it atomic.
What we built	OPS Ed25519-signs a fingerprint of the execution it approved. A custom Reth executes the candidate provisionally, compares, and commits only on a match. Divergence excludes the whole transaction — nonce, fee, writes.
Does it work	Yes. Cross-org redirection, caught reverts, delegatecall storage tricks and read-only probes are all rejected on a real node with real OPS and PostgreSQL.
What it costs	+10% to +20% of Reth's block-building time per transaction. On this laptop that is 1.2 to 14 µs per transaction.
Biggest gap	The guarantee holds only on <i>our</i> producer. It is not a consensus rule, and it does nothing for confidentiality.

1 The problem

OPS traces a transaction with `debug_traceCall`, walks every internal call frame, checks each target against the org graph in PostgreSQL, then forwards it. The check is sound about the trace. It is not sound about the transaction.

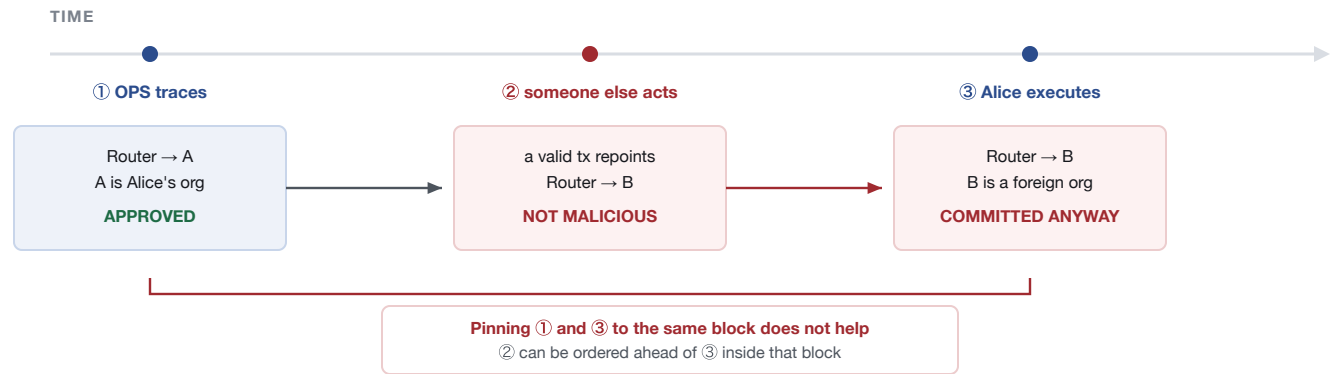


Figure 1. The race, reproduced on a test chain. No stale cache is needed and nobody has to be attacking — an ordinary transaction that happens to land first is enough.

Everything the proxy can do about it, and why it is not enough

MITIGATION	CEILING
Re-trace at latest / pending before forwarding	Shrinks the window. Same-block reordering lives entirely inside it.
Shorten or disable the trace cache	Removes one divergence source, creates no atomicity.
Higher fees, private mempool	Influences ordering. Cannot freeze state or block context.
Redact the receipt afterwards	Cannot undo a state transition that already committed.
Enforce in the participant's own RPC node	Cannot stop an upstream producer from committing.

THE GUARANTEE WE ARE TRYING TO BUY

A governed transaction commits only if its **actual execution** satisfies the policy chosen for it. Otherwise it is rejected. Guaranteed success or inclusion by a deadline is a separate promise and is *not* claimed.

A related option — write the policy into Solidity — was rejected because these are customer contracts. They know nothing about our policies and must not be edited, wrapped or redeployed. A voluntary check protects only the paths that remember to call it. A node-side check is mandatory for every transaction the producer builds.

2 What we built

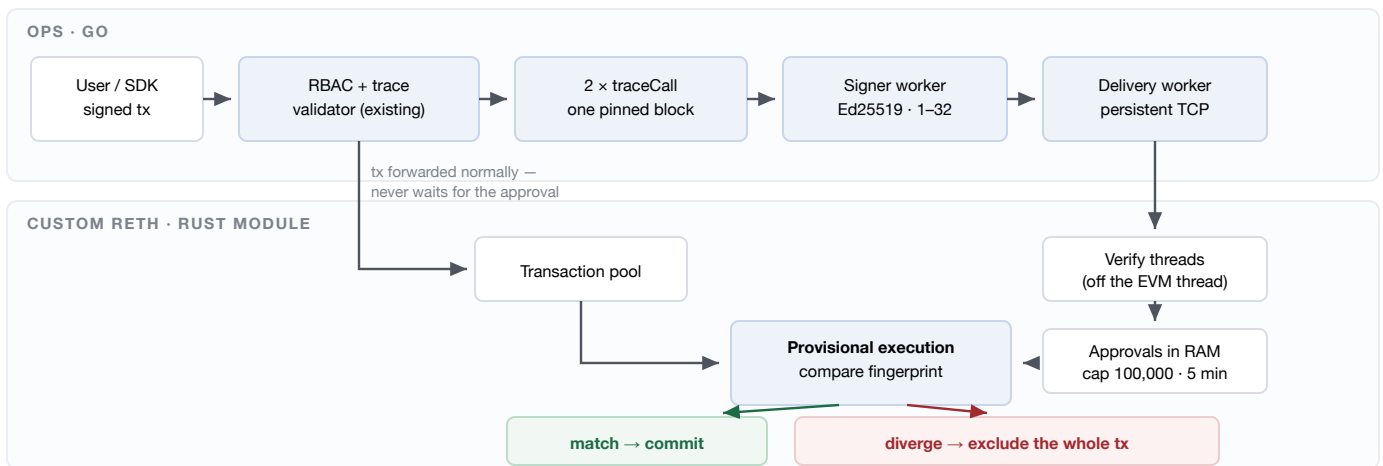


Figure 2. Approval and transaction travel separately and neither waits for the other. The HTTP handler returns as soon as the node accepts the transaction.

What the approval actually contains

Not a copy of the policy — a fingerprint of the approved execution, bound to the exact transaction hash. The mode is part of the signed material, so it cannot be downgraded on the wire or by the node.

COMPARED BEFORE COMMIT	STRICT V2 (original)	CALLS V3 (default now)
Call order, parent, caller, destination, executed code, storage context	exact	exact
Full calldata, ETH value, success/revert of every frame incl. caught ones	exact	exact
Storage values, balances, contract nonces, return data, logs	exact (fees normalized out)	may differ
CREATE / CREATE2 / SELFDESTRUCT	exact	automatic strict-V2 fallback

WHY V3 HAD TO EXIST

Strict V2 breaks on contention. The first approved write to a shared counter makes every other approval stale.

Measured: **98 transactions updating one counter** → **1 completed, 97 stuck pending**. Dropping storage/logs/return-data equality fixed it: the same test now runs **64 of 64**, including through the real OPS HTTP service.

Decisions worth recording

DECISION	WHY, AND WHAT IT BOUGHT
Compute the fingerprint directly	The first version built Geth prestate/diff reports, converted them to JSON, built a second canonical JSON tree, hashed that, and re-hashed bytecode the node already had a hash for. Removing it cut 82–84% of the module's added block time.
Keep Ed25519, add batching	HMAC is 46× faster (0.70 µs vs 32.40 µs) but lets anyone holding the node's key forge an OPS approval. One signature over 1–32 approvals keeps OPS-only signing and amortizes verification to ~1.2 µs.
Sign immediately, never wait to fill	A fill timer would cost the oldest approval ~6 ms at 5,000/s. Consequence: batches are mostly small — in a live run 256 approvals formed 229 batches, 204 of them singletons.
Crypto off the EVM thread	Execution thread does a lookup and a comparison, nothing else.
Binary wire format	120 bytes per approval, 4-byte framing, <code>TCP_NODELAY</code> . JSON kept only for compatibility tests.
Upstream Reth untouched	Built through public extension traits at the pinned commit. Standard RPC and transaction formats unchanged; approvals use a separate private port.

Operating limits

BEHAVIOUR	VALUE
Wait for a missing approval	5 s from pool insertion, on its own timer; other transactions are not blocked. Then dropped from the pool.
Readiness check	At the next payload build — a late approval may miss one candidate and wait for the next.
Bounds (fail closed on overflow)	100,000 stored approvals · 4,096 outgoing queue · 32 inbound connections · 16 KiB frames
After node restart	Approval memory is empty. No automatic re-creation or resend; the client resubmits.
Code size	2,760 lines Rust · 1,836 lines Go · +136/–10 lines in existing OPS files

3 Proof it enforces

The three-step demo, on real OPS + PostgreSQL + Redis + custom Reth

#	SCENARIO	RESULT
1	Alice calls Bank A: Router → Relay → Vault	OPS approves from real DB rules, Reth includes it, receipt returns through OPS.
2	Alice makes a nested call into Bank B	OPS refuses before forwarding. <i>This already worked.</i>
3	OPS approves Alice's Bank A call; a valid higher-fee transaction repoints the relay to Bank B first	Reth excludes Alice's transaction. Nonce, balance and application writes unchanged. <i>This is the case the proxy cannot catch.</i>

Bypasses attempted, on a real node

ATTEMPT	OUTCOME
Same-block target change redirects an approved call into a foreign org	excluded
Application catches the forbidden call's revert and returns success	outer success does not override the veto
Foreign-contract call with no storage writes and an identical return value	rejected
DELEGATECALL — and, after review, CALLCODE — into foreign code using permitted storage	rejected; storage context is bound
Forged signature, tampered batch member, downgraded or unknown hash mode	rejected; every mutated byte is covered
Changed nested arguments, ETH value, call count or bytecode under a calls approval	rejected, nothing committed
A different DID replays the same signed transaction	denied by real OPS before the node

Validation totals

SUITE	RESULT
Real-node integration scenarios	17 → 20 → 35 node + 12 OPS HTTP → 19 call-hash; 55 in the strict-V2 regression set
Rust unit tests	18, incl. a shared Go/Rust golden vector and 128 generated combinations
Go tests	approval, RBAC and tracer suites under <code>-race</code> , repeated 20×
Old-vs-new fingerprint equivalence	46 identical fingerprints, including all 8 recorded denials
Determinism control	all 27 block triples identical across off / strict / calls — 81 blocks
Lint and build	clippy, rustfmt, gofmt, release build — clean

4 Performance

SCOPE OF EVERY NUMBER BELOW

Measured on Ivan's MacBook — Apple M2 Max, 12 logical cores, 32 GB RAM, macOS 26.6.1 arm64, release builds, PostgreSQL and Redis in local Docker, all nodes on loopback with no peers.

The Reth tables time one region: the real `default_ethereum_payload` — execution plus state and receipt roots — divided by transactions included. Excluded: pool admission, both preflight traces, signing, delivery, signature verification, the approval wait, Engine API import.

This is not TPS. A producer-path percentage cannot be extrapolated to a system throughput target.

Overhead in Reth

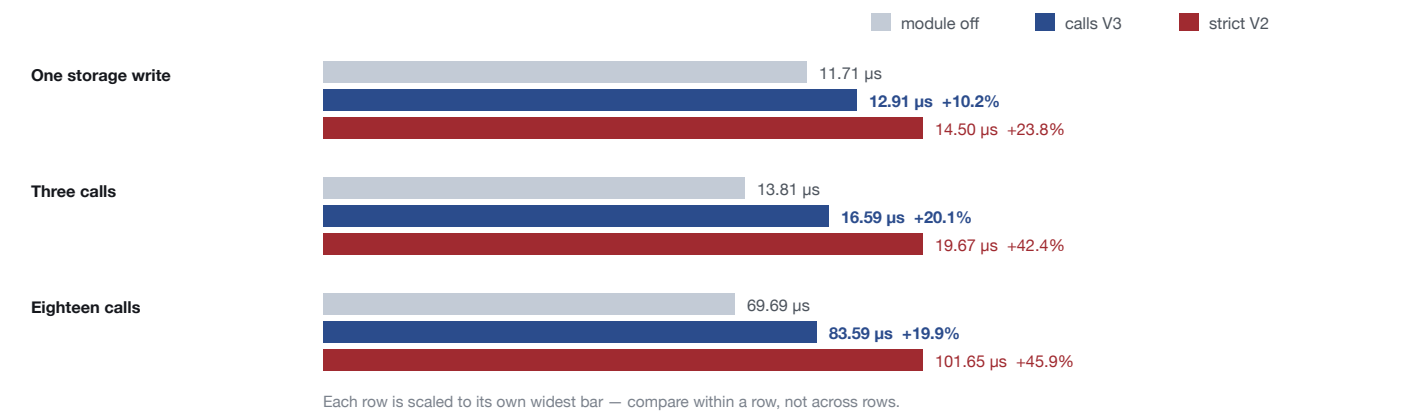


Figure 3. Median μ s per transaction. Same binary in all three modes, 9 measured blocks per mode per workload, 32 transactions per block, rotated mode order, one discarded warmup per node. Calls V3 is both safer under contention and 11–18% cheaper than strict.

Where the overhead went

The optimization that mattered — historical V1 fingerprint, same method

WORKLOAD	STOCK RETH	FIRST VERSION	AFTER FIX	LEFT VS STOCK
One storage write	11.54 μ s	24.79 μ s	13.72 μ s	+18.9%
Three calls	14.00 μ s	45.55 μ s	19.61 μ s	+40.1%
Eighteen calls	69.97 μ s	199.63 μ s	92.77 μ s	+32.6%

Stage profile, eighteen calls — what actually got deleted

STAGE — MEDIAN MICROSECONDS PER TX	BEFORE	AFTER
Prepare/encode fingerprint, dispose reports	109.51	3.75
Keccak of the final fingerprint	12.88	12.67
EVM execution with call tracing	31.75	31.68
Build Geth call frames	2.25	2.29

Instrumented run, includes clock overhead — do not add these to the table above. No upstream Reth bug: the first version simply used helpers meant for building debug-trace responses inside the enforcement path.

Cryptography — outside the block builder, not free

OPERATION	COST PER APPROVAL
Ed25519 verify_strict, predecoded (microbenchmark)	32.40 μ s
Ed25519 at the Reth ingress, before batching	$\approx$ 50 μ s
One signature over a full batch of 32, amortized	1.19–1.84 μ s
Live OPS run, singleton-heavy traffic	$\approx$ 39 μ s
Go signing: single / full batch of 32	19 μ s / 29 μ s
HMAC-SHA-256, prepared key (rejected alternative)	0.70 μ s

Asynchrony overlaps this with execution; it does not remove the CPU cost. Never add these to the block-building tables as sequential latency.

Approval delivery latency

In the live run, 92 of 256 approvals were already present when Reth first saw the transaction. The other 164 arrived **0.116 ms** after pool insertion on average, **0.422 ms** at worst — all under half a millisecond. This is permission availability, not block inclusion.

Whole-stack, through real OPS

Short local HTTP bursts · 16 concurrent clients, 64 tx/block

	BASELINE OPS	SIGNED-APPROVAL PATH
Submissions per second	607–771	557–695
Median request latency	≈17 ms	≈21 ms

Gasstorm, calls V3, 120 s adaptive browser runs · fresh stack per case · every confirmation re-checked against a unique Reth receipt

WORKLOAD	ENFORCEMENT	UI PEAK	BEST 30 S	RECEIPTS	UNCONFIRMED	FAILED
ETH transfer	off	2,216	2,045	209,675	225	0
ETH transfer	on	2,032	1,801	193,936	171	0
ERC20 approve	off	1,980	1,951	211,124	166	0
ERC20 approve	on	1,635	1,469	163,828	914	0

Cost of enforcement, from the runs above

WORKLOAD	RECEIPTS	BEST 30 S THROUGHPUT
ETH transfer — almost no call frames to hash	–7.5%	–11.9%
ERC20 approve — a real contract call	–22.4%	–24.7%

Identical settings on both sides of each pair: 500 tx/s initial, 50 tx/s step, 2,000 target pending, 120 s, ten wallets, one-second blocks, 200M gas. One run per case, each on its own fresh stack, so the consecutive-run nonce collision in §5 does not apply. **Zero submission failures and zero duplicate confirmations in all four.** Peaks are the dashboard's retained maximum; "best 30 s" counts canonical block transactions in the best window with no net pool growth and can include draining a backlog, so it does not prove a sustained maximum. The ERC20 on-run left 914 transactions without a receipt against 166 for its baseline. The load generator warns that ten wallets model out at ~150 TPS, so this fixture is driven far past its intended shape. Measured with a second, idle stack resident on the same laptop.

Stop / Reset controls, enforcement on · measured in the earlier strict-V2 sessions

CHECK	RESULT
Constant 2,000 tx/s offered for 75 s	150,293 sent · 149,205 confirmed · 0 failed
Stop → Reset	245 ms → 233 ms
Requested 6,000 tx/s for 20 s	49,120 sent · 44,756 confirmed · 0 failed — reached ~2,027 tx/s, not 6,000
Stop → Reset after overload, then retest	247 ms → 241 ms · 9,898 confirmed · 0 failed

BOTTOM LINE

Inside Reth the module costs 1.2 to 14 µs of extra block-building time per transaction, or 10 to 20% of a small baseline. **Through the whole stack** the cost depends entirely on how much the transaction actually calls: 7.5% of receipts on a bare ETH transfer, 22.4% on an ERC20 approve. Quote the workload, never a single number. Signature verification costs tens of microseconds per approval on separate threads, unless batches fill — which under real traffic they mostly do not. Nothing here was measured on production hardware, at production concurrency, or for longer than two minutes.

5 What is *not* solved



Figure 4. A producer that does not run the module is outside the guarantee entirely. Network-wide enforcement needs deterministic, replayable authorization rules in block validation — a much larger piece of work, never in scope here.

Architectural gaps — these are not "finish the work" items

GAP	DETAIL
Producer-only	Enforcement is wired into our own block building. Import and historical execution use the stock executor. Not a consensus rule; a bypassing producer is explicitly outside the guarantee.
No confidentiality	The node still holds every org's data in plaintext in one shared state trie. This changes what can be <i>written</i> , not what an operator can see. No ZK, no ASRN operator privacy.
Calls V3 allows real divergence	Same calls, different internal accounting, passes. A regression test changes an internal increment from 0 to 120 and confirms calls mode allows it. State-dependent business rules need strict mode — which brings back the contention problem.
No parameter masks	You cannot say "ignore this one argument". It is all-exact or all-relaxed, per category.

Open defects

DEFECT	STATUS
Nonce collisions across consecutive runs	A second load test on the same wallets collides with transactions left queued by the first. A later ERC20 run recorded 1,652 submission failures out of 26,302 , with 2,863 nonces overlapping previously discarded transactions. Resetting the UI does not clear Reth's pool. Repeated-run reliability is not established. The investigation document other reports link to does not exist yet — only the raw data.
Engine driver timeout	One interactive 2,000 tx/s run: the test Engine driver timed out on <code>engine_forkchoiceUpdatedV2</code> , the launcher tore down the stack, the disconnected UI stayed stuck on "test in progress". Root cause unresolved.
ERC20 approval timeouts	One enabled run logged 39 approval timeouts and stalled near the end. The 100,000-entry store is a likely contributor but there is no capacity-rejection counter, so the causal chain is unverified.
Nonce gaps without the module too	A baseline run with enforcement off also lost a nonce with 2,472 transactions queued behind it. A load-generator sender bug was reproduced and patched in a test overlay, never deployed; the specific missing transaction was never traced end to end.

Missing product behaviour

MISSING	EFFECT
Rejection notification	A vetoed transaction returns a hash and then simply never gets a successful receipt. The client cannot tell refusal from delay, and the excluded transaction may sit in the pool for a later attempt.
Retry ownership	OPS does not re-create or resend. After a restart the approval memory is empty and the client must resubmit.
Revocation protocol	The 5-minute sweep is memory hygiene, not policy revocation.
Ordering independence	A dropped transaction stalls that sender's later nonces. Other senders are unaffected.

Out of scope by design · and fixture limits

ITEM	NOTE
Policy DSL / Solidity policies	Track A, separate worktree. This PoC uses the existing PostgreSQL Go rules unchanged.
Business predicates	Approvals carry execution fingerprints, not policy. No way to express "allow if the counter is below N".
Transport security	Plain TCP in a local perimeter. The signature authenticates each approval; it does not encrypt the channel. Persistent mTLS was measured as the alternative and not implemented.
Transaction formats	Protected legacy and EIP-1559 only · max 128 call frames · max 1 MiB canonical data.
Version coupling	OPS and the Reth module upgrade together. No V1 fallback; an older node rejects V3 approvals outright.
Identity and infrastructure	All identities, orgs, wallets and keys are fixtures. Login uses <code>mockauth</code> ; no external identity provider ran in any test. Blocks come from a small test Engine driver, not a consensus client. No indexer, no explorer UI.

WHAT THE MEASUREMENTS DO NOT ESTABLISH

Not maximum or sustained throughput (longest run: 120 s) · not production hardware · not the whole stack (headline % covers block building only) · not repeated-run reliability · not a prediction for any target TPS.

6 Next

#	ACTION	WHY NOW
1	Fix the repeated-run nonce collision	Until it is closed, no load result from a second consecutive run can be quoted.
2	Decide the transport	Persistent mTLS if the node only needs authenticated delivery; keep Ed25519 batches if nodes must be unable to forge approvals. Both measured, neither deployed across perimeters.
3	Measure the whole chain once	Admission → both traces → sign → deliver → verify → wait → import, on representative hardware. Every current number excludes most of that.
4	Add a rejection signal	So a client learns it was vetoed instead of inferring it from a missing receipt.
5	Specialize the tracer	The general call tracer still sits in the interpreter's inspection path. Next plausible saving — measure it, do not assume it.
6	Decide if producer-only is enough	Or scope the consensus-level version.

Running it

```
python3 poc/reth-signed-approvals/setup.py
export OPS_RETH_BINARY="$PWD/.tmp/approval-target/release/ops-reth-approvals-poc"
export TEST_DATABASE_URL='postgres://postgres:postgres@127.0.0.1:5432/ops_approvals_test?sslmode=disable'

python3 poc/reth-signed-approvals/run.py test          # correctness scenarios
python3 poc/reth-signed-approvals/run.py bench        # Reth producer benchmark
bash poc/reth-signed-approvals/demo.sh story --pause  # the 3-step demo
bash poc/reth-signed-approvals/gasstorm.sh ui         # browser load test → 127.0.0.1:18000/load-test/
```

Needs Rust, Go, Python 3, `cast`, `solc` 0.8.35, Docker and a dedicated PostgreSQL 15. Enable in OPS with `OPS_APPROVAL_TARGET` + `OPS_APPROVAL_SEED_FILE` ; on the node with `OPS_APPROVALS=1`, `OPS_APPROVAL_LISTEN`, `OPS_APPROVAL_PUBLIC_KEY`, `OPS_APPROVAL_CHAIN_ID`. `OPS_APPROVAL_HASH_MODE=strict` selects the old fingerprint. Raw data, logs and manifests are in `poc/reth-signed-approvals/evidence/`.

Appendix • Screenshots

Unaltered native browser captures of the Gasstorm dashboard, cropped only at the bottom. Every run routes through OPS — **Through Privacy Proxy** is selected in each. Gas and fill charts read zero because OPS redacts block gas usage.

> GasStorm

Dashboard

Load Test

History

Privacy

Docs

L1L2LoadGen

Load Test30.00s • 838 tx/s peak • 99.7% confCompleted

Mode

Direct

Through Privacy Proxy

JWT

Paste privacy-proxy JWT

Optional locally (token auto-refreshes). For an external/prod proxy, log in there and paste a fresh JWT.

Gasless

Off

Gasless network

Skips funding and sends 0-value transactions — for chains with zero gas fees.

Pattern

Constant

Ramp

Spike

Adaptive

Mixed

Realistic

Adaptive rate that finds maximum throughput

Transaction type

Storage Write ~43k gas

Write to contract storage

Requires contract deployment (auto-deployed on first test)

Initial (tx/s)

500

Step (tx/s)

50

Target pending

1000

Duration

30

s (30s)

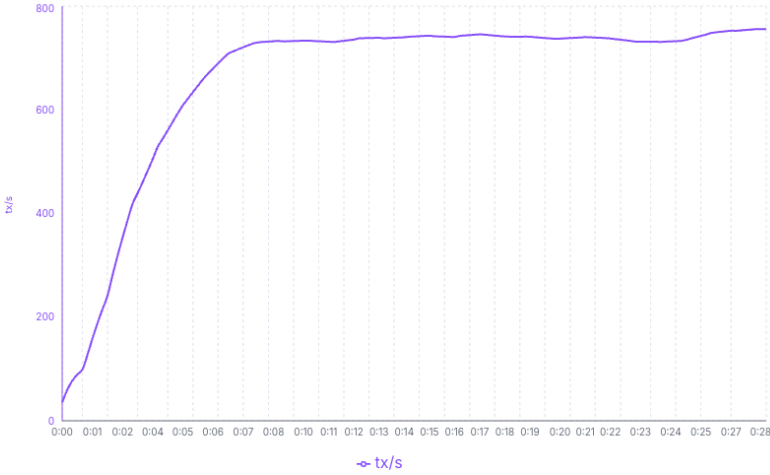
Fix nonce gaps

Completed • 30.00s • 838 tx/s peak • 99.7% conf • 25,368 txs

Reset

TPS Over Time

Last: 836.6 tx/sPeak: 838.0 tx/s



Test Results

THROUGHPUT

Avg0.0 Mgas/sPeak0.0 Mgas/s

Avg TPS848Peak TPS838

BLOCKS

Block—Avg Block—

Fill0.0%Blocks30

GAS

Base Fee—Gas Price—

Total Gas0.00 MgasTotal Tx25,368

PROVENANCE

AttestationOFFHSM Provider—

HSM Key—HSM FailoverOFF

Transaction Verification ⚠️

25,453Sent

25,368Confirmed

0Failed

85Pending

Confirmation Rate

Average TPS

99.7%

848 tx/s

⚠️ 85 transactions still pending

A1 • Calls V3 solves the shared-state problem. Storage Write, 30 s, enforcement on: **25,368 confirmed, 0 failed**, 85 pending, 838 tx/s peak. Every transaction writes the same storage slot — the exact workload strict V2 could not handle.

GasStorm

DashboardLoad TestHistoryPrivacyDocs

L1L2LoadGen

Load Test2m 0.0s • 2216 tx/s peak • 99.9% confCompleted

ModeDirectThrough Privacy Proxy

JWT

Optional locally (token auto-refreshes). For an external/prod proxy, log in there and paste a fresh JWT.

GaslessOffGasless network

Skips funding and sends 0-value transactions – for chains with zero gas fees.

PatternConstantRampSpikeAdaptiveMixedRealistic

Adaptive rate that finds maximum throughput

Transaction type

ETH Transfer~21k gas

Simple native ETH transfer

Initial (tx/s)500Step (tx/s)50Target pending2000

Duration120 s (2m 0s) ☐ Fix nonce gaps

Completed • 2m 0.0s • 2216 tx/s peak • 99.9% conf • 209,675 txsReset

TPS Over TimeLast: 2011.6 tx/s Peak: 2216.0 tx/s

Test Results

THROUGHPUT

Avg0.0 Mgas/sPeak0.0 Mgas/s

Avg TPS1749Peak TPS2216

BLOCKS

Block—Avg Block—

Fill0.0%Blocks120

GAS

Base Fee—Gas Price—

Total Gas0.00 MgasTotal Tx209,675

PROVENANCE

AttestationOFFHSM Provider—

HSM Key—HSM FailoverOFF

Transaction Verification

209,900Sent

209,675Confirmed

0Failed

225Pending

Confirmation Rate99.9%

Average TPS1749 tx/s

225 transactions still pending

A2 - Baseline, enforcement off. ETH transfer, 120 s adaptive: 2,216 tx/s peak, **209,675 confirmed, 0 failed**, 225 without a receipt at the final check. Control for A3, captured on its own fresh stack.

> GasStorm

DashboardLoad TestHistoryPrivacyDocs

L1L2LoadGen

Load Test2m 0.0s • 2032 tx/s peak • 99.9% confCompleted

ModeDirectThrough Privacy Proxy

JWT

Paste privacy-proxy JWT

Optional locally (token auto-refreshes). For an external/prod proxy, log in there and paste a fresh JWT.

GaslessOffGasless network

Skips funding and sends 0-value transactions – for chains with zero gas fees.

PatternConstantRampSpikeAdaptiveMixedRealistic

Adaptive rate that finds maximum throughput

Transaction type

ETH Transfer~21k gas

Simple native ETH transfer

Initial (tx/s)Step (tx/s)Target pending

500502000

Duration120s (2m 0s)Fix nonce gaps

Completed • 2m 0.0s • 2032 tx/s peak • 99.9% conf • 193,936 txsReset

TPS Over TimeLast: 1452.4 tx/s Peak: 2032.0 tx/s

THROUGHPUT

Avg0.0 Mgas/s

Avg TPS1617

Peak0.0 Mgas/s

Peak TPS2032

BLOCKS

Block

Fill0.0%

Avg Block

Blocks119

GAS

Base Fee

Total Gas0.00 Mgas

Gas Price

Total Tx193,936

PROVENANCE

AttestationOFF

HSM Key

HSM Provider

HSM FailoverOFF

Transaction Verification

194,107Sent

193,936Confirmed

0Failed

171Pending

Confirmation Rate

Average TPS

99.9%

1617 tx/s

171 transactions still pending

A3 • Same workload, calls V3 enforcement on. 2,032 tx/s peak, **193,936 confirmed**, **0 failed**, 171 without a receipt. Against A2 that is 7.5% fewer receipts and an 8.3% lower peak. Every field above the chart matches A2 — same adaptive ramp, same ten wallets, same duration — and both runs verified every confirmation against a unique successful Reth receipt.

